

Implementation Details

AI-Assisted Programming Platform for Education and Assessment

1. Introduction

This document explains the implementation details of the AI-Assisted Programming Platform for Education and Assessment. The platform combines a browser-based programming environment, a Docker-isolated code execution sandbox, a two-stage AI Mentor (a 30-billion-parameter mentor model audited by a 3-billion-parameter validator model), automated grading, rubric-based teacher review, analytics, and a structured tutorial library.

The platform provides separate interfaces for students and teachers. Students can access programming assignments, solve problems in a browser-based coding environment, run and submit code, receive AI mentor support, review tutorials, track their progress, and study feedback flashcards. Teachers can create and manage assignments, enroll students, manage the question bank, review submissions, generate rubrics, grade student work, and analyze student performance.

In addition to the standard teacher role, the system includes a higher-privileged teacher role referred to as the **Coordinator Teacher**. The Coordinator Teacher is not a full system administrator, but has additional academic management privileges such as approving teacher registrations, viewing all students, and assigning students to other teachers.

A key purpose of the system is to support AI-assisted learning while also protecting academic integrity. For this reason, the platform includes an **Exam Mode** lockdown mechanism that disables AI Mentor access, hides assistive panels, and gates submissions to a teacher-defined time window during exam-style assignments.

2. System Overview

The system is a full-stack web application deployed as a set of Docker Compose services. Users access the platform through a browser and select the appropriate portal according to their role. The system separates the learning and assessment workflows into role-based modules.

The main platform capabilities are:

- Role-based student and teacher portals
- JWT-based authentication and authorization
- Programming assignment management
- Student enrollment by group or individually
- Question bank management
- Multi-file, multi-language code editor (Monaco)
- Docker-isolated code execution and automated testing
- Interactive WebSocket terminal with clickable error-line links
- Submission history tracking
- Context-aware AI Mentor support with two-stage validation
- Teacher-controlled Exam Mode lockdown
- Student analytics and learning progress tracking
- Feedback flashcards generated from mistakes and feedback

- Rubric-based grading
- AI-assisted rubric generation
- AI-assisted grading suggestions
- AI-assisted question variation generation
- Block-based tutorial library (~97 C tutorials in 8 categories / 23 sub-categories) with per-section PDF export
- Class-level analytics
- Teacher registration approval by the Coordinator Teacher
- Student-to-teacher assignment management by the Coordinator Teacher

3. Technology Stack

The platform is implemented with a modern full-stack JavaScript/TypeScript architecture. Concrete framework and runtime choices are listed below.

3.1 Frontend

The frontend is a single-page application built with **React 18** (Vite-bundled), **Material UI 7** for components and theming, **react-router 7** for routing, the **Monaco editor** for the code editor (the same engine used by VS Code), **xterm.js** with the fit addon for the interactive terminal, **recharts** for analytics charts, **marked** + **DOMPurify** for safe markdown rendering, and **jsPDF** for client-side PDF export of tutorial sections.

The student coding workspace includes an IDE-like editor that supports multiple files per problem (EditorTabBar) and displays language-specific source files such as `main.c`, `main.py`, `main.js`, `Main.java`, `Program.cs`, or `main.cpp`.

3.2 Backend

The backend is a **Node.js + TypeScript** service running on the **Express** framework. Database access is handled through the **Prisma ORM** against a PostgreSQL database. Mentor streaming responses are delivered over Server-Sent Events at `/api/ai/chat/stream`. The backend is responsible for authentication, authorization, assignment operations, problem management, submission handling, code execution requests, AI Mentor communication, analytics, grading, and privileged teacher operations.

3.3 Database

The system uses **PostgreSQL** for persistent storage. The schema is defined and versioned in `prisma/schema.prisma` with fifteen committed migrations applied automatically at container start. The database stores users, roles, teacher-student relationships, student groups, problems, test cases, assignments, enrollments, submissions, AI interactions, rubrics, grades, feedback flashcards, and system-level settings such as exam mode.

3.4 Code Execution Layer

The code execution layer is a **Docker-based runner** implemented in `backend/src/services/dockerRunner.ts`. Each Run/Submit spawns an ephemeral container (`--rm`) for the appropriate language image and pipes the student's source through `stdin/stdout`. This layer replaced an earlier Judge0/isolate-based runner that was incompatible with `cgroup v2` on the host. The full design is described in Section 16.

3.5 AI Integration

AI inference is provided by **Ollama** serving two open-weights models locally on an NVIDIA L4 GPU (23 GB VRAM):

- **qwen3-coder:30b** (~20 GB VRAM) - the mentor model. Responsible for hints, explanations, and Socratic guidance.
- **qwen2.5:3b-instruct** (~2.4 GB VRAM) - the validator model. Audits each mentor reply against the academic-integrity policy.

Both models are pinned in VRAM with `OLLAMA_KEEP_ALIVE=-1`. The full two-stage architecture is described in Section 17.

Summary of the technology stack:

Layer	Technology	Purpose
Frontend framework	React 18 + Vite	Single-page application bundle
UI components	Material UI 7	Design system and theming
Editor	Monaco editor	Multi-language code editor
Terminal	xterm.js + fit addon	Interactive program I/O
Charts	recharts	Analytics visualisations
Markdown	marked + DOMPurify	Safe markdown rendering
PDF export	jsPDF	Tutorial section export
Backend runtime	Node.js + TypeScript	API service
Backend framework	Express	HTTP routing
ORM	Prisma	Database access and migrations
Database	PostgreSQL	Persistent storage
Sandbox	Docker (Compose)	Code execution isolation
LLM runtime	Ollama	Local model serving
Mentor model	qwen3-coder:30b	Socratic hint generation
Validator model	qwen2.5:3b-instruct	Policy audit of mentor replies
Auth	JWT bearer tokens	Session management
Deployment	Docker Compose on GCE VM (L4 GPU)	Orchestration

4. User Roles and Permissions

The platform uses role-based access control. Each user can only access the pages and actions allowed for their role.

4.1 Student

Students can:

- View assigned homework, practice tasks, and exams
- Open programming problems
- Write code in the online editor (multi-file)
- Run code and view compiler/runtime output in an interactive terminal
- Submit solutions for automated evaluation
- View their submission history
- Ask the AI Mentor for help during normal learning activities
- Use the Debug-Prints shortcut to ask the mentor for instrumentation
- Access tutorials and export sections as PDF
- View personal analytics
- Review feedback flashcards

Students cannot manage assignments, view other students' submissions, edit question bank content, approve teachers, or access teacher management pages.

4.2 Standard Teacher

Standard teachers can:

- Create and manage assignments for their students
- Enroll students or groups into assignments
- Create and edit programming problems
- Manage test cases (public and hidden) and starter code
- Use AI-assisted problem generation tools
- Create or generate grading rubrics
- Review and grade submissions
- View analytics for their assigned students
- Enable Exam Mode for relevant assessment scenarios

4.3 Coordinator Teacher

The Coordinator Teacher is a higher-privileged teacher role designed for a department coordinator or course coordinator. The Coordinator Teacher is not a full administrator. The Coordinator Teacher can:

- Access all standard teacher features
- Approve or reject teacher registration requests
- View all students in the system
- Assign students to other teachers
- Manage teacher-student distribution
- Monitor broader course-level or department-level teaching activity

5. Authentication and Authorization

Authentication is implemented with **JWT bearer tokens**. The platform starts with a portal selection screen where users choose either the Student Portal or the Teacher Portal. Each portal provides a role-specific sign-in flow.

After successful login, the backend issues a signed JWT containing the user's role. The frontend stores the token in browser storage and sends it as an `Authorization: Bearer <token>` header on every

API request. Each protected route runs an Express middleware that decodes the token, validates the role, and rejects the request if the user is not permitted to perform the action.

Student navigation includes: Dashboard, Assignments, Analytics, Flashcards. Teacher navigation includes: Dashboard, Assignments, Students, Question Bank, Grading, Analytics, and (for Coordinator Teacher only) Approvals.

6. Student Portal Implementation

6.1 Student Dashboard and Navigation

The Student Portal provides a learning-focused interface. Students can move between assignments, analytics, flashcards, and AI Mentor-supported programming activities. The navigation is designed to keep the main learning features easily accessible.

6.2 Assignment Listing

Students can view programming tasks assigned to them. Assignments are categorized as homework, practice, or exams. Each assignment exposes title, topic tags, difficulty level, supported programming language, deadline, and completion status. The list also surfaces aggregate progress (how many of the assigned tasks have been completed).

6.3 Problem-Solving Workspace

The problem-solving workspace is one of the core student features. It is divided into three main areas:

1. Assignment and tutorial panel: switches between available problems and the block-based tutorial library.
2. Problem and editor panel: shows the problem description, examples, language selection, source file tabs, and the Monaco editor.
3. AI Mentor panel: contextual chat with the two-stage mentor pipeline (collapses to single-column in Exam Mode).

The center panel includes: problem title, difficulty level, programming language, problem statement, example input/output, code editor with multi-file tabs (EditorTabBar), Run button, Submit button, an interactive xterm.js terminal area, and submission history.

The editor supports per-language starter files. A C problem opens with a `main.c` template, a Python problem with `main.py`, a Java problem with `Main.java`, C# with `Program.cs`, JavaScript with `main.js`, and C++ with `main.cpp`. Students may add, rename, or close additional source files within the same problem.

6.4 Run Workflow

The Run workflow allows students to test their code interactively before submitting it for grading.

4. The student writes or edits code in the browser.
5. The student clicks Run.
6. The frontend opens a WebSocket session for the terminal and POSTs the source code, language, and active file list to the backend.
7. The backend spawns a Docker container for the selected language image via `dockerRunner.ts`.
8. Compilation (if needed) runs first, with a 30-second timeout.

9. On success, the binary runs interactively; stdin/stdout/stderr are bridged through the WebSocket to xterm.js.
10. Compiler messages and stack traces are scanned for line numbers; matches become clickable links that jump the editor to the offending line.

6.5 Submit Workflow

The Submit workflow evaluates the student's solution against the assignment's test cases.

11. The student submits the current code.
12. The backend retrieves all test cases for the problem (public and hidden).
13. Using the compile-once-run-N session model, dockerRunner.ts compiles the code once and runs the resulting binary against each test case in sequence.
14. For every test case the runner records stdout, stderr, exit code, and wall-clock execution time.
15. The platform compares actual output with expected output for each test case.
16. Per-test results plus the overall status (accepted / failed / compile_error / runtime_error / time_limit) are stored in the submission record.
17. The student sees per-test results and a refreshed submission history.

Submission history rows store status, programming language, total execution time, timestamp, related assignment/problem, and the full source code at submission time. This historical record supports analytics, grading, and flashcard generation.

6.6 Compiler and Runtime Feedback

Compiler and runtime feedback is rendered directly inside the student interface. The terminal uses **xterm.js** with a custom link provider that turns patterns like `line 14, main.c:23`, or stack-trace file references into clickable links. Clicking a link moves the editor cursor to the referenced line.

If a student writes an assignment operator where a comparison was intended, the compiler error appears in the terminal panel and the student can either fix it directly or ask the AI Mentor for an explanation through the chat panel.

6.7 AI Mentor

The AI Mentor is integrated into the problem-solving workspace. It receives contextual information about the current problem, the student's code, recent run output, the last 20 conversation turns, and (when available) the editor cursor context. The mentor can:

- Provide hints calibrated to the current hint level (1 = nudge, 5 = explicit walkthrough without code)
- Explain programming concepts
- Ask Socratic guiding questions
- Explain compiler and runtime errors
- Review parts of the student's code without rewriting it
- Reply in English or Turkish (detected via `mentor.ts:detectMessageMode`)
- Insert temporary debug prints around the cursor via the Debug-Prints button

Every mentor reply passes through a second-stage validator before reaching the student. The full architecture is described in Section 17.

6.8 Exam Mode Behaviour for Students

Exam Mode is a full lockdown of the student workspace, not just a flag on the AI panel. When Exam Mode is active, the frontend (StudentWorkspace.jsx + AppHeader.jsx):

- Removes the AI Mentor panel and chat input from the DOM
- Hides the assignments panel and the submissions history sidebar
- Collapses the workspace into a single-column layout
- Hides administrative navigation items and the notifications bell in AppHeader (lockdown prop)
- Installs a beforeunload guard that warns on accidental tab/window close
- Installs a contextmenu guard that disables right-click
- Gates submissions to the teacher-defined start/end time window
- Shows the 'Finish Exam' button only after every visible test passes

On the backend, the AI Mentor route checks exam state on every request and rejects mentor calls during an active exam window. This dual enforcement (frontend hides, backend blocks) prevents bypass via direct API calls.

6.9 Tutorials

The Tutorials module ships with approximately **97 C tutorials** organised under 8 main categories (C Tutorial, C Functions, C Files, C Structures, C Enums, C Memory, C Errors, C More) and 23 sub-categories. Tutorials are stored as JSON files under backend/src/data/tutorials/c/ and served via a 3-level GET /api/tutorials/index endpoint that returns category - subCategories - topics.

Each tutorial follows a **block-based schema** that mirrors w3schools' interleaved layout. The block types are:

Block type	Renders as
text	Markdown-rendered paragraph
code	Monaco read-only editor with syntax highlighting
syntax	Monaco editor with a 'SYNTAX' label
note	Yellow Alert callout (InfoOutlined icon)
goodtoknow	Blue Alert callout (LightbulbOutlined icon)
list	Bulleted list with inline markdown items

The TutorialModal component renders both the new block-based schema and the legacy body/code schema so older tutorials continue to work. Students can export any tutorial section as a PDF (jsPDF) directly from the side panel.

6.10 Student Analytics

The Student Analytics page gives students a personal overview of their learning progress. Metrics include total submissions, success rate, number of solved problems, number of AI hints used, learning trend, activity history, error breakdown, and progress over time.

6.11 Feedback Flashcards

The Flashcards module converts previous feedback, code issues, and learning suggestions into reusable study cards. Cards are generated by an LLM pass with the following constraints (flashcardService.ts):

- format: "json" mode, temperature 0.1, num_predict 3000
- Target card count tuned to difficulty: 2 / 3 / 5 / 7 cards
- Base timeout 180s, +30s per attempt, capped at 8 minutes
- Post-filters: isValidCard() (schema check), dedupeCards() (Jaccard similarity), hasGrounding() (every card must quote a specific token from the student's code)
- Prompt includes explicit GOOD/BAD examples to anchor card quality

Each card contains feedback type, related problem, explanation of the issue, the problematic code snippet, a suggested improvement, and a before/after code comparison. Examples include redundant code, inefficient algorithms, incorrect output formatting, missing edge case handling, and cleaner code suggestions.

7. Teacher Portal Implementation

7.1 Teacher Dashboard

The Teacher Dashboard provides a high-level overview of teaching activity: total students, total attempts, accepted submissions, AI hints used, assignment summaries, due dates, assignment status, and assignment modes.

7.2 Exam Mode Control

Teachers activate Exam Mode per assignment. The setting is stored on the assignment row and consulted by both the frontend lockdown logic (Section 6.8) and the backend AI Mentor route. Exam Mode can be applied to all enrolled students, specific student groups, or selected assessment scenarios. Frontend lockdown plus backend AI-call blocking together preserve academic integrity.

7.3 Assignment Management

The Assignments page lets teachers create, edit, publish, and manage assignments. The assignment form is restructured so the problem selector is the first field; selecting a problem auto-fills the title, description, and language from the linked problem (AssignmentModal.jsx). The form captures: title, linked problem, description, due date, enrolled students, status, mode, visibility rules, and (when applicable) exam start and end times.

Enrollment is integrated into the assignment form via the EnrollModal component running in 'select' mode (no API calls until save), so a teacher can build an assignment and its roster in a single dialog.

7.4 Assignment Modes

The system supports two assignment modes: homework/practice and exam. In homework or practice mode, students can access problems and use AI Mentor support. In exam mode, the teacher defines a controlled assessment window: students are restricted by exam start and end times, AI Mentor access is disabled, and the frontend enters lockdown.

7.5 Student Enrollment

Teachers enroll students into assignments using groups, individual selection, or a combination of both. The flow supports selecting complete student groups, selecting individual students, combining group and individual selection, viewing selected student count, and saving enrollment changes atomically.

8. Coordinator Teacher Features

8.1 Teacher Registration Approval

When a new teacher registers, the Coordinator Teacher reviews the request from the Approvals page and approves or rejects it. This prevents unauthorized teacher accounts from gaining access to teaching features.

8.2 Global Student Visibility

Unlike a standard teacher (who sees only assigned students), the Coordinator Teacher can list every student in the system.

8.3 Assigning Students to Teachers

The Coordinator Teacher can assign students to standard teachers, supporting workflows where a department coordinator distributes students across instructors or teaching assistants.

8.4 Difference Between Coordinator Teacher and Administrator

The Coordinator Teacher is not a full administrator. The role is focused on academic coordination rather than system administration. It cannot edit user identities, alter system flags directly, or manage infrastructure - those would require a true admin role.

9. Question Bank Implementation

9.1 Question Bank Management

The Question Bank stores reusable programming problems. Teachers can search, filter, create, edit, and manage questions. Each question can include: title, problem description, difficulty level, topic tags, supported languages, starter code, hidden reference solution, public test cases, hidden test cases, rubric information, and attempt statistics.

9.2 Problem Creation

Teachers create new programming questions using a structured form covering metadata, description, starter code, reference solution, and test cases. The hidden reference solution is used for teacher-side validation and is never sent to students. Test cases can be configured as visible or hidden. Public test cases help students understand expected behaviour; hidden test cases improve assessment reliability.

9.3 AI-Assisted Question Generation

Teachers can request the LLM to generate easier, similar, or harder versions of an existing problem. The generated question appears in a preview pane for the teacher to review, edit, reject, or approve before it is added to the question bank. This keeps the teacher in control while reducing the time required to create new assessment material.

10. Rubric Implementation

Rubrics allow teachers to grade submissions in a structured and consistent way. A rubric can include multiple criteria such as correctness, algorithm efficiency, edge case handling, code readability, use of appropriate programming constructs, and output formatting. Each criterion has a name, description, maximum points, and a scoring guide.

The platform also supports AI-assisted rubric generation. Teachers can generate a rubric suggestion for a selected problem and then approve or modify it before use.

11. Student Management and Progress Tracking

The Students page allows teachers to view student information and learning progress. Student records include name, email, academic year, assigned teacher, group membership, completed problems, and progress percentage.

Teachers can search and filter students and open individual student details to review deeper performance information: attempts, hints used, solved languages, difficulty distribution, language performance, problem breakdown, and top errors.

Standard teachers see only their assigned students; the Coordinator Teacher sees all students and can assign students to teachers.

12. Grading Implementation

The Grading page allows teachers to evaluate student submissions through this workflow:

18. Teacher selects an assignment.
19. The system lists enrolled students and their submission statuses.
20. Teacher opens a student's submission.
21. The system displays code, per-test results, runtime, and overall status.
22. Teacher scores the submission using rubric criteria.
23. Teacher optionally writes comments.
24. Teacher saves the final grade.

The grading screen distinguishes public and hidden test results so teachers can see whether a solution passes only the visible examples or also the hidden evaluation cases.

12.1 AI-Assisted Grading Suggestions

The AI Suggest feature proposes rubric scores and comments based on the submitted code, test results, rubric criteria, output correctness, and code-quality indicators. The teacher remains responsible for the final grade - AI suggestions are decision support, not automatic grading.

12.2 Exporting Grades

The grading module exports grading data as an Excel file using the SheetJS (xlsx) library. This helps teachers keep offline records or submit grades to external systems.

13. Class Analytics Implementation

The Class Analytics page provides class-level performance information rendered with recharts: number of students, total submissions, accepted submissions, class accept rate, number of problems, weekly

submission activity, hardest problems, accept rate by problem, and common error types. For the Coordinator Teacher, analytics can also support broader monitoring across multiple groups or teachers.

14. Database Design

The database schema is defined in `prisma/schema.prisma` and migrated automatically at container start (15 migrations applied non-interactively by `prisma migrate deploy`). Main logical entities:

- User
- Role
- Student profile
- Teacher profile
- Teacher registration request
- Teacher-student assignment
- Student group
- Problem
- Test case (public/hidden flag)
- Assignment
- Assignment enrollment
- Submission
- Per-test execution result
- AI interaction log
- Feedback flashcard
- Rubric
- Rubric criterion
- Grade
- Exam mode setting
- System flag

14.1 User and Role Data

The user model stores authentication-related information (hashed password, email, display name) and role assignment. Roles define whether a user is a student, standard teacher, or Coordinator Teacher.

14.2 Problem and Test Case Data

Problems store the problem statement, difficulty, topic tags, supported languages array, starter code per language, and the hidden reference solution. Test cases are linked to problems and carry a public/hidden flag plus stdin/expected-stdout.

14.3 Assignment and Enrollment Data

Assignments link problems to students or groups. The enrollment row determines which students can access a given assignment and tracks per-student completion status.

14.4 Submission Data

Each submission stores the student, assignment/problem, language, submitted code, overall status, total execution time, per-test results, and timestamp.

14.5 AI Data

AI interactions store the student question, the mentor response, hint level, validator verdict, validator violation list, generated flashcards, and a timestamp. This table powers analytics, hint counting, and flashcard generation.

14.6 Grading Data

Grades are linked to submissions and rubric criteria, allowing per-criterion scoring and structured teacher feedback.

15. Backend Workflow Details

15.1 Authentication Workflow

25. User submits login credentials.
26. Backend validates the credentials.
27. Backend identifies the user role and issues a signed JWT.
28. JWT is returned to the frontend and stored client-side.
29. Frontend renders the correct portal and navigation based on the JWT role claim.
30. Every subsequent API request carries the JWT in the Authorization header; backend middleware verifies it.

15.2 Assignment Workflow

31. Teacher creates or edits an assignment via AssignmentModal.
32. Teacher selects a problem from the question bank (title/description/language auto-populate).
33. Teacher defines assignment metadata and mode.
34. Teacher enrolls students or groups using EnrollModal in 'select' mode.
35. On save, the backend stores the assignment and writes all enrollment records in a single transaction.
36. Students see only the assignments available to them.

15.3 Code Execution Workflow

37. Student writes code in the editor.
38. Student clicks Run or Submit.
39. Frontend sends source code, language, and active file list to the backend; for Run it also opens a WebSocket for the terminal.
40. Backend validates access to the assignment.
41. Backend invokes dockerRunner.ts, which spawns the language-appropriate Docker container with --rm.
42. Container compiles or interprets the code.
43. For Submit, the runner reuses the compiled binary across all test cases (compile-once-run-N session).
44. Per-test stdout/stderr/exit-code/wall-time are collected.
45. Backend stores submission and per-test results.
46. Frontend displays output, errors, test status, and execution time.

15.4 AI Mentor Workflow

47. Student sends a message to `/api/ai/chat/stream` (Server-Sent Events).
48. Backend checks whether AI access is allowed (Exam Mode would block here).
49. Backend builds the mentor prompt from: assignment text, current student code, recent run status + stdout/stderr, last 20 chat turns, hint level, locale, and (optionally) [FOCUSED CODE NEAR CURSOR] from the editor.
50. Mentor model (qwen3-coder:30b) streams a draft reply.
51. Validator pipeline runs the draft through heuristic checks (validator.ts) and, if needed, the small validator model (qwen2.5:3b-instruct).
52. If the reply is rejected, the backend either repairs it or returns a safe fallback hint.
53. Approved reply is streamed to the student; the interaction is stored for analytics and flashcard generation.

15.5 Teacher AI Workflow

54. Teacher requests AI support for problem generation, rubric generation, or grading suggestion.
55. Backend collects the relevant problem, submission, or rubric context.
56. AI generates a suggestion (JSON-mode for rubrics/cards).
57. Teacher reviews the suggestion.
58. Teacher approves, edits, or rejects the suggestion - nothing is saved without explicit approval.

15.6 Coordinator Teacher Workflow

59. Coordinator Teacher reviews teacher registration requests.
60. Coordinator Teacher approves or rejects each request.
61. Coordinator Teacher views all students.
62. Coordinator Teacher assigns students to appropriate teachers.
63. Standard teachers then manage their assigned students.

15.7 API Surface (selected)

The backend exposes a REST API plus one streaming endpoint. Selected routes:

Method	Path	Purpose
POST	<code>/api/auth/login</code>	Issue JWT
POST	<code>/api/auth/register</code>	Student/teacher registration
GET	<code>/api/assignments</code>	List assignments for the current user
POST	<code>/api/assignments</code>	Create assignment (teacher)
POST	<code>/api/assignments/:id/enroll</code>	Enroll students/groups
GET	<code>/api/problems/:id</code>	Fetch problem detail
POST	<code>/api/run</code>	Run code (interactive, WebSocket)
POST	<code>/api/submit</code>	Run all test cases, store submission
GET	<code>/api/submissions</code>	Submission history

Method	Path	Purpose
POST	/api/ai/chat/stream	Mentor chat (SSE stream)
POST	/api/ai/flashcards	Generate feedback flashcards
POST	/api/ai/rubric	Generate rubric suggestion
POST	/api/ai/grade	Generate grading suggestion
POST	/api/ai/variation	Generate problem variation
GET	/api/tutorials/index	3-level tutorial TOC
GET	/api/tutorials/:tag/:lang	Tutorial content (blocks)
GET	/api/analytics/student/:id	Per-student analytics
GET	/api/analytics/class	Class-level analytics
POST	/api/grades	Save rubric grade
GET	/api/grades/export	Excel export
GET	/api/coordinator/approvals	Pending teacher registrations
POST	/api/coordinator/approvals/:id	Approve/reject

16. Code Execution Sandbox

Code execution is implemented as a Docker-based runner in `backend/src/services/dockerRunner.ts`. This section describes the design and the migration that led to it.

16.1 Why Docker (not Judge0 / isolate)

An earlier iteration of the platform used the open-source **Judge0** backend, which relies on **isolate** for sandboxing. **isolate** v1.x is built on **cgroup** v1, which is no longer the default on modern Linux hosts. The host VM ships with **cgroup** v2 only, and **isolate** v1 fails to create sandboxes in that environment - producing `time-limit-exceeded` errors on every submission regardless of the program. Rewriting **isolate** to support **cgroup** v2 was out of scope, so the runner was rebuilt around Docker itself.

16.2 Architecture

Each `Run/Submit` call ends in `runInDocker()` or `createDockerSession()` (the `compile-once-run-N` variant used by `Submit`). Both spawn an ephemeral container with the `--rm` flag, mount the student's source files into `/code`, pipe `stdin/stdout/stderr` through Node streams, and tear down the container when the process exits. The backend has a `stdin` error handler in place to absorb EPIPE failures cleanly.

Per-language images and compile commands:

Language	Image	Compile / run command
Python	python:3.12-slim	python3 main.py

Language	Image	Compile / run command
JavaScript	node:20-slim	node main.js
C	gcc:13	gcc -O2 -o /code/bin /code/main.c && /code/bin
C++	gcc:13	g++ -O2 -o /code/bin /code/main.cpp && /code/bin
Java	eclipse-temurin:21-jdk-alpine	javac Main.java && java Main
C#	mcr.microsoft.com/dotnet/sdk:8.0-alpine	dotnet build --nologo -o /code/bin /code/app.csproj && dotnet /code/bin/app.dll

16.3 Timeouts and Resource Limits

- `COMPILE_TIMEOUT_MS` = 30_000 (30 seconds for compile/build steps).
- `DEFAULT_RUN_TIMEOUT_MS` = 15_000 (15 seconds per test case).
- Containers run with `--rm` so they self-clean after exit.
- The runner sends `SIGKILL` to the docker process on timeout to ensure timely teardown.
- `stdin` is closed via Node's pipe so well-behaved programs receive EOF cleanly.

16.4 Compile-Once-Run-N Session

For Submit, the runner compiles the source one time and then runs the resulting binary against each test case. This collapses N compile+run cycles into 1 compile + N runs, materially reducing submission latency for compiled languages (C, C++, Java, C#).

16.5 Interactive Terminal

Run mode wires the container's `stdin/stdout/stderr` through a WebSocket to **xterm.js** on the client. `InteractiveTerminal.jsx` registers a custom link provider (`term.registerLinkProvider`) that matches common error-line patterns - `line N`, `file.ext:N`, language-specific stack traces - and turns them into clickable links. Clicking a link calls the parent's `onErrorLineClick` which jumps the Monaco editor to the matching line.

17. AI System Architecture

This section describes the AI Mentor's internal architecture. It is the single most differentiated component of the platform and is not adequately covered by a single 'send context to LLM' diagram.

17.1 Two-Stage Tutor / Student Loop

Mentor replies are produced by a two-stage pipeline:

64. Stage 1 - Mentor (qwen3-coder:30b) generates a draft reply from the assembled prompt.
65. Stage 2 - Validator audits the draft against the academic-integrity policy. If the draft passes, it is forwarded to the student; if it fails, it is either repaired or replaced with a safe fallback hint.

The split exists because mentor models are good at producing helpful text but bad at policing themselves. A small, fast validator is cheaper than expecting a large model to be its own judge and

improves consistency under pressure (long sessions, exam-like prompts, or attempts to extract the solution).

17.2 Heuristic Floor + AI Escalation

The validator (`backend/src/services/validator.ts`) implements a two-tier check:

- Tier 1 - Heuristic floor. Cheap regex/pattern checks that catch the obvious violations without an LLM call: banned phrases (negation-aware), explicit exact-fix directives, runtime guessing, assignment walkthroughs, solution-seek leaks, overly long responses, and others. This tier short-circuits clear-cut rejections and is fast enough to run on every reply.
- Tier 2 - AI validator (qwen2.5:3b-instruct). Invoked only when the heuristic floor is undecided. Issues a JSON verdict (approve / repair / reject) plus a reason.

Heuristic floor highlights:

- Banned phrase detection is negation-aware via `bannedPhraseIsAsserted()` - the mentor saying 'I won't write the complete solution' does not trip the rule, but the mentor saying 'here is the complete solution' does.
- `EXACT_FIX_PATTERNS` uses a negative lookbehind to avoid false positives on noun-form 'change' (e.g. 'minimal change needed to fix' is fine; 'change line 14 to ...' is not).
- 8-second timeout on the AI validator with graceful fallback to the heuristic verdict if the small model is slow.

17.3 Mentor Quality Assessment

`backend/src/services/mentorQuality.ts` runs in parallel with the validator and flags low-quality replies even when they are policy-compliant. Detections include:

- `transcript_artifact` - the model leaked role labels or system text
- `generic_fallback` - the reply is a non-specific encouragement with no engagement
- `echoes_question` - the reply restates the student's question (Jaccard similarity ≥ 0.82)
- `repeats_previous_reply` - the reply is too similar to a prior mentor turn (Jaccard ≥ 0.72)
- `ignores_error_context` - the student's run output is non-empty but the reply does not reference it
- `ignores_focused_line` - the student provided cursor context but the reply does not address it

17.4 Mentor Prompt Composition

The mentor prompt is assembled in `buildMentorPrompt()` from these sections in order:

66. System role and global rules (no solutions, hint-level discipline, Socratic style).
67. Locale directive ('Respond in Turkish (Türkçe). Use natural Turkish prose; keep code, error messages, and API names in their original form.' when applicable).
68. Assignment text.
69. Current student code (all open files).
70. [EXECUTION CONTEXT] block summarising the last run: `compile_error` / `runtime_error` / `wrong_answer` / `accepted`, plus `stdout/stderr`.
71. [FOCUSED CODE NEAR CURSOR] block (when the student provides editor context): a +/-5 line window centred on the cursor with a `>` marker on the active line.
72. [CONVERSATION SO FAR] block: last 20 turns of chat, labelled Student/Mentor, with an instruction to build on prior context.

73. [STUDENT MESSAGE] - the current question.

74. Mode-specific override (HINT MODE includes the current hint level).

Ollama is called with `num_ctx 16384` to fit prompt + history + reply comfortably; the validator uses `num_ctx 4096`.

17.5 Conversation Memory

The mentor does not store conversation state server-side per session. Instead, the frontend (ProblemPage.jsx) holds the chat log in React state and forwards a sanitised snapshot on every request:

- Last 20 entries (10 user/assistant turns).
- Excludes the greeting message and any still-streaming bubble.
- Excludes failed-request error bubbles ([error] ...).
- Sent as `conversationHistory: [{role, content}]`.

This gives the mentor working memory across a session without requiring a stateful conversation store on the backend.

17.6 Hint-Level Progression

Hints escalate through discrete levels: a light nudge at level 1, a more pointed observation at level 2, a partial walkthrough at level 4, and an explicit non-code walkthrough at level 5. The hint level is tracked per problem on the frontend and is included in HINT MODE prompts so the mentor knows how much to reveal.

17.7 Locale Handling

`detectMessageMode()` and `normalizeMentorLocale()` select between English and Turkish based on the student's message and an explicit locale hint. Turkish locale activates: bilingual fallback messages, Turkish-language banned-phrase patterns, and a system directive that keeps code/errors/API names in their original form while switching the prose to Turkish.

17.8 Debug-Prints Shortcut

The student workspace exposes a Debug-Prints button next to Run/Submit. Clicking it: (1) captures the editor cursor and the +/-5 line window via `getCursorContext()`; (2) chooses a language-appropriate print call (`printf`, `print`, `console.log`, `std::cout`, `System.out.println`, `Console.WriteLine`); (3) sends a pre-written request to the mentor asking for a few temporary trace prints around the cursor line, with explicit instructions not to change the program logic; (4) the mentor returns a small snippet the student can paste back. This teaches print-debugging without the mentor giving away the answer.

17.9 Model Sizing and VRAM Budget

Both models are loaded concurrently in the L4's 23 GB VRAM:

Model	VRAM	Context	Role
qwen3-coder:30b	~19.4 GB	16384	Mentor
qwen2.5:3b-instruct	~2.4 GB	4096	Validator
Free	~1.2 GB	-	KV cache headroom

Models are pinned with `OLLAMA_KEEP_ALIVE=-1` so they survive idle time. `OLLAMA_FLASH_ATTENTION=1` is set to reduce KV cache memory. Cold start of the 30B model is approximately 30-60 seconds (disk-bound) and is hidden from users by a warm-up container that issues a one-token request to each model on Compose startup.

18. Security and Academic Integrity

The platform combines several mechanisms to improve security and academic integrity:

- Role-based authentication and authorization (JWT, per-route middleware).
- Separate student and teacher portals at the routing layer.
- Restricted Coordinator Teacher privileges (no infrastructure access).
- Hidden reference solutions (never sent to students).
- Hidden test cases (separated public/hidden flag).
- Exam start and end time gating, enforced server-side.
- AI Mentor route blocks calls during Exam Mode.
- Exam Mode frontend lockdown (single column, no AI panel, beforeunload + contextmenu guards).
- Dual enforcement: hiding UI is not enough; the backend blocks API calls too.
- Two-stage validator rejects mentor replies that leak solutions even when prompted aggressively.
- Submission history is immutable and timestamped.
- Teacher-controlled grading (AI suggestions never apply automatically).
- Teacher registration approval workflow.
- Docker isolation of every student execution.

Exam Mode is the most important academic-integrity feature: AI assistance is available during practice while disabled, hidden, and dual-enforced during exams.

19. Testing and Validation

The system is validated through functional tests covering the main workflows.

Student-side scenarios:

- Student login
- Assignment listing
- Problem selection
- Code editing (single and multi-file)
- Run operation (interactive terminal)
- Submit operation (compile-once-run-N)
- Compiler error display with clickable error-line links
- Submission history update
- AI Mentor hint request (validator approved)
- AI Mentor solution-seeking attempt (validator rejected)
- AI Mentor restriction during Exam Mode (UI hidden + API blocked)
- Debug-Prints shortcut
- Tutorial browsing through the 3-level TOC
- Tutorial section PDF export
- Analytics display

- Flashcard review

Teacher-side scenarios:

- Teacher login
- Assignment creation (problem-first form auto-fill)
- Assignment editing
- Student enrollment (groups + individuals)
- Exam Mode activation
- Question creation
- Test case management (public/hidden)
- AI-generated question suggestion
- Rubric creation
- AI-generated rubric suggestion
- Submission review
- Rubric-based grading
- AI grading suggestion
- Grade export (Excel)
- Class analytics display

Coordinator Teacher scenarios:

- Viewing all students
- Assigning students to teachers
- Reviewing teacher registration requests
- Approving teacher registrations
- Rejecting teacher registrations

20. Challenges and Solutions

20.1 Balancing AI Support and Academic Integrity

Challenge: AI assistance is useful for learning, but unrestricted access during exams could harm academic integrity.

Solution: Two-stage AI Mentor (mentor + validator) prevents solution leaks even during normal learning. Exam Mode provides dual enforcement: the frontend hides the AI panel and locks the workspace, and the backend rejects mentor calls during the exam window.

20.2 Replacing the Code Execution Sandbox

Challenge: The original Judge0 backend depended on isolate v1, which is incompatible with the host's cgroup v2 - every submission failed with TLE regardless of correctness.

Solution: A new Docker-based runner (dockerRunner.ts) replaced Judge0. Each Run/Submit spawns an ephemeral container with `--rm`, per-language images, explicit timeouts, and a compile-once-run-N session model that collapses N compile+run cycles into 1 compile + N runs for compiled languages.

20.3 Supporting Different Teaching Responsibilities

Challenge: A single teacher role is not enough for all academic workflows because some instructors may need broader coordination privileges.

Solution: The platform includes a Coordinator Teacher role that can approve teacher registrations, view all students, and assign students to other teachers without being a full administrator.

20.4 Supporting Multiple Programming Languages

Challenge: Programming assignments may use different languages: C, C++, Python, Java, C#, or JavaScript.

Solution: The platform stores supported languages per problem, picks the matching Docker image at runtime, and the frontend renders language-specific starter files in the editor tab bar.

20.5 Making Feedback Reusable

Challenge: Students may forget compiler errors, AI explanations, and teacher feedback after a session.

Solution: Flashcards are generated from each session with strict post-filters (isValidCard, dedupeCards, hasGrounding) so every card quotes specific tokens from the student's own code, and explicit GOOD/BAD examples in the prompt anchor card quality.

20.6 Reducing Teacher Workload

Challenge: Creating problems, rubrics, and grading submissions requires significant time.

Solution: AI-assisted tools help teachers generate problem variations, rubric suggestions, and grading suggestions. Nothing is applied without explicit teacher approval.

20.7 Migrating the Tutorial Corpus

Challenge: The original tutorial schema (body + trailing code) could not represent w3schools-style interleaved text and code, and rendering quality suffered.

Solution: A block-based schema (text / code / syntax / note / goodtoknow / list) was introduced, ~97 C tutorials were migrated into it (with a 3-level category/sub-category/topic hierarchy), and TutorialModal renders both new and legacy schemas. Per-section PDF export was added via jsPDF.

21. Deployment Topology

The platform is deployed on a GCE virtual machine with an attached NVIDIA L4 GPU (23 GB VRAM, 15 GiB host RAM, no swap). All services run under **Docker Compose**:

Service	Image / runtime	Role
app-backend	Node + TypeScript (Express + Prisma)	REST + SSE API
app-frontend	Vite-built React static bundle	Web UI
postgres	Postgres	Persistent database
ollama	Ollama (GPU-enabled)	Local LLM serving
ollama-warmup	curl one-shot	Pre-warms both models on Compose up so first user does not wait for cold load

The Ollama service mounts the GPU via the NVIDIA container runtime (deploy.resources.reservations.devices or the shorter gpus: all form). Backend reaches Ollama over the

internal Docker network at `http://ollama:11434`. Prisma migrations are applied non-interactively at backend container start. Compose volumes persist Postgres data and the Ollama model cache so neither has to be rebuilt across restarts.

22. Conclusion

The AI-Assisted Programming Platform for Education and Assessment provides a complete environment for programming education. It supports students with online code editing, interactive Docker-isolated execution, automated testing, a two-stage AI Mentor (qwen3-coder:30b mentor audited by qwen2.5:3b-instruct validator), a 97-tutorial block-based learning library, analytics, and feedback flashcards. It supports teachers with problem-first assignment management, question bank tools, group + individual enrollment, exam control, rubric-based grading, AI-assisted evaluation, and class analytics.

The system also includes a Coordinator Teacher role for broader academic management. This role can approve teacher registrations, view all students, and assign students to teachers, while still remaining separate from a full administrator role.

Overall, the implementation provides a practical balance between AI-supported learning and controlled assessment. AI is available during practice and homework to guide students through a Socratic, validator-audited mentor, while Exam Mode disables AI assistance, hides assistive panels, and gates submissions during exams to protect academic integrity.